

Criando o projeto

Meu Blog

Neste módulo vamos construir, do zero, um blog completo: artigos, comentários, curtidas, favoritos, métricas e recomendações. Nesta primeira aula, preparamos a base: um Supabase novo, o projeto já conectado, o briefing, a identidade visual, o login e o mais importante para um blog: **quem pode fazer o quê**, com papéis guardados de forma segura.

Supabase meu-blog

Projeto conectado

Knowledge

Login e perfis

Papéis separados

Autor × leitor

Formação Claude Code 2026 · Continua o módulo 28 (Integração Lovable e Supabase) · Baseado em docs.lovable.dev e supabase.com/docs, consultados em setembro de 2026 · Telas: reprodução ilustrativa

O blog *em dez aulas*

AULA 01 • HOJE

Projeto, login, perfis e papéis

AULA 02

Página de criação de artigos

AULA 03

Página de visualização dos artigos

AULA 04

Comentários I: comentar

AULA 05

Comentários II: editar, apagar, responder

AULA 06

Comentários III: moderação

AULA 07

Curtidas

AULA 08

Favoritos

AULA 09

Dashboard analítico

AULA 10

Artigos relacionados e em alta

O QUE JÁ SABEMOS (MÓDULO 28)

Supabase próprio, conexão sem Enable Cloud, migrations aprovadas, RLS, login com perfis, dono definido pelo banco e teste com dois usuários. Aqui vamos mais rápido nessas partes.

O QUE É NOVO

Papéis (autor e admin), conteúdo com rascunho e publicação, interações de vários usuários no mesmo artigo e **métricas agregadas**.

Quem pode *fazer o quê*

Antes de qualquer tela, a matriz de permissões. Ela guia cada regra de RLS das próximas aulas.

AÇÃO	VISITANTE	LEITOR LOGADO	AUTOR	ADMIN
Ler artigos publicados e comentários	✓	✓	✓	✓
Comentar, curtir e favoritar	—	✓	✓	✓
Escrever, editar e publicar os próprios artigos	—	—	✓	✓
Ver métricas dos próprios artigos	—	—	✓	✓
Moderar comentários de qualquer artigo	—	—	só nos seus	✓
Dar ou tirar papéis	—	—	—	pelo painel

"Leitor logado" é o papel padrão de quem cria conta. Autor e admin são concedidos, nunca escolhidos no cadastro.

Um Supabase novo: *meu-blog*

1 Na organização do curso, crie o projeto `meu-blog` em **South America (São Paulo)**, com senha forte salva no gerenciador (aula 03 do módulo 28).

2 Convide para a equipe os e-mails de teste que vai usar (autor e leitores), por causa do e-mail padrão do Supabase.

3 Depois de ter o endereço do preview, ajuste **Site URL** e **Redirect URLs** (padrão `localhost:3000`).

POR QUE UM PROJETO NOVO

Blog e estante são apps diferentes, com usuários e regras diferentes. Misturar tabelas num banco só complicaria as políticas e o limite de dados de cada um.

Limite do Free: 2 projetos ativos. Com `minha-estante` e `meu-blog` , você está no limite. Para um terceiro, pause um deles no painel do Supabase.

Projeto conectado *e Knowledge*

CRIANDO JÁ CONECTADO

- 1 No painel do Lovable: + na caixa de prompt → **Databases** → `meu-blog`.
- 2 A organização do curso já está vinculada desde o módulo 28.
- 3 **Nada de Enable Cloud.** Confira em More → Cloud: aparece `meu-blog`.

O `meu-blog` não deve aparecer com o ⚠ de "já usado por outro projeto". Cada app, seu banco.

Knowledge (Settings → Knowledge):

Meu Blog: blog pessoal com artigos, comentários, curtidas e favoritos.

- Português do Brasil.
- Backend: Supabase próprio (meu-blog). NÃO usar Lovable Cloud.
- Papéis: leitor (padrão), autor, admin. Papéis ficam na tabela "papeis", NUNCA em perfis; verificação sempre no banco.
- Toda tabela com RLS. O banco define o dono dos dados (auth.uid()).
- Visual editorial: fundo #FAFAF7, texto #1B1B1F, azul #3446D9; títulos em Newsreader, texto em Inter.
- Celular primeiro. Sem dados inventados: estados vazios honestos.

As quatro perguntas, *versão blog*

1 Quem usa? Visitantes que leem; leitores que interagem; uma autora que escreve.

2 O que precisam fazer? Ler com conforto, comentar, curtir, guardar favoritos; a autora escreve, publica e acompanha números.

3 Quais dados? Perfis, papéis, artigos, comentários, curtidas, favoritos e visualizações.

4 O que não pode acontecer? Leitor virar autor sozinho, rascunho vazar, alguém apagar o comentário de outro, favoritos de um leitor ficarem visíveis para outros.

A RESPOSTA 4 VIRA REGRA NO BANCO

- Papéis numa tabela que o usuário **não** altera
- Rascunho só visível para o autor
- Comentário: só o dono edita; moderação por papel
- Favoritos: leitura só do dono

Cada aula do módulo vai fechar um desses itens com uma política de RLS e um teste com mais de uma conta.

Papéis *em tabela separada*

Se o papel ficasse numa coluna de `perfis`, e o leitor pode editar o próprio perfil, bastaria mudar "leitor" para "autor". Por isso os papéis ficam numa tabela à parte, que o app **não consegue alterar**.

TABELA	QUEM LÊ	QUEM ALTERA
<code>perfis</code> id, nome, bio	Todos (nome aparece nos artigos e comentários)	O próprio usuário
<code>papeis</code> user_id, papel	O próprio usuário (para a tela saber o que mostrar)	Ninguém pelo app: só você, pelo painel do Supabase

Aqui a leitura pública de perfis é segura: o blog mostra o nome de quem escreve e comenta. E-mail continua no esquema `auth`, não exposto.

UMA FUNÇÃO NO BANCO

As políticas das próximas aulas vão perguntar "este usuário é autor?". Uma **função no banco** responde consultando `papeis`, e todas as regras usam a mesma função. Assim a verificação fica no servidor, nunca só na tela.

Esconder o botão "Escrever" de quem não é autor é só conforto. Quem impede um leitor de publicar é a política da tabela de artigos (aula 02), usando essa função.

O prompt *da base do blog*

Prompt (Plan mode):

Monte a base do Meu Blog (veja o Knowledge):

1. Cabeçalho: logo, Início, Sobre; botão Entrar; no celular, botão Menu.
2. Início: título, subtítulo e "Artigos recentes" com estado vazio.
3. Login com e-mail e senha (Supabase Auth), confirmação de e-mail e "esqueci a senha".
4. Tabela perfis (id ligado ao usuário, on delete cascade; nome, bio) criada por gatilho no cadastro. Leitura pública; cada um altera só o próprio.
5. Tabela papeis (user_id, papel: autor ou admin). O usuário lê só o próprio papel; NINGUÉM insere, altera ou apaga pelo app. Crie uma função no banco "tem_papel" para as políticas futuras.
6. Se o usuário for autor: botão "Escrever", link "Meus artigos" e selo "autor".

NA MIGRATION, CONFIRA

- ✓ RLS ativo em `perfis` e `papeis`
- ✓ `papeis` sem nenhuma política de INSERT, UPDATE ou DELETE
- ✓ Papel restrito a "autor" ou "admin"
- ✓ Gatilho de perfis testado com um cadastro logo depois

Sem papel na tabela, o usuário é leitor. Não é preciso guardar "leitor": ausência de papel já significa isso.

Tornando-se *autora*

- 1 Crie a conta da **Autora Teste** pelo app e confirme o e-mail.
- 2 No Supabase: **Authentication** → **Users**, copie o id dela.
- 3 No **Table Editor**, em **papeis**, insira uma linha: esse id e o papel **autor**.
- 4 Recarregue o app logado como ela: aparecem "Escrever", "Meus artigos" e o selo "autor".

POR QUE PELO PAINEL

Dar papel é uma decisão administrativa rara. Fazer isso no painel, e não por uma tela do app, elimina toda uma categoria de risco: não existe botão para ser explorado.

O painel do Supabase ignora o RLS porque você é administrador (aula 07 do módulo 28). É exatamente o que permite inserir o papel que o app não consegue.

Visitante × *autora*



Visitante · reprodução ilustrativa



Autora Teste logada · reprodução ilustrativa

MESMA PÁGINA, DOIS ESTADOS

A tela muda conforme o papel lido de **papeis**.

ESTADO VAZIO HONESTO

"Nenhum artigo publicado ainda", e para a autora, o caminho: Escrever.

IDENTIDADE EDITORIAL

Newsreader nos títulos, muito espaço e um único azul de destaque.

No celular *e com três contas*



390 px · reprodução ilustrativa

REGISTRO DOS TESTES

EXEMPLO

Meu Blog · base

- ✓ visitante vê Entrar, sem Escrever
- ✓ leitor logado, sem Escrever
- ✓ autora Escrever + selo autor
- ✓ perfis 3 linhas (gatilho ok)
- ✓ papeis 1 linha (autora)
- ✓ leitor → papel via app recusado
- ✓ celular menu presente

bookmark: "blog base v1"

Prompt (Chat mode) · conferência:

Liste as políticas RLS de perfis e papeis.
Um leitor logado consegue inserir ou alterar
uma linha em papeis? Não altere nada.

Erros comuns *na base do blog*

Papel dentro de perfis. O usuário edita o próprio perfil e vira autor. Papéis em tabela separada, sem escrita pelo app.

Verificar papel só na tela. Esconder o botão não impede publicar. A verificação é no banco, pela função.

"Escolha seu papel" no cadastro. Todo mundo escolheria autor. Papel é concedido, não escolhido.

Reaproveitar o Supabase da estante. Mistura usuários e regras de dois apps e sobrescreve segredos.

Esquecer o limite de 2 projetos. No Free, um terceiro projeto exige pausar um dos anteriores.

Testar só como autora. Os erros de permissão aparecem com a conta de leitor e com o visitante.

Teste o que *ficou*

1. Por que os papéis ficam fora da tabela perfis?

A) Por organização

B) O usuário altera o próprio perfil e poderia se promover

C) O Supabase exige

D) Para economizar espaço

B. Em papeis, o app não tem permissão de escrita.

2. Como a Autora Teste virou autora?

A) Escolheu no cadastro

B) Uma linha em papeis, inserida pelo painel do Supabase

C) Editou o perfil

D) Pediu no chat do app

B. Papel é concedido por quem administra, fora do app.

3. Quem realmente impede um leitor de publicar?

A) O botão escondido

B) A política da tabela de artigos, usando a função de papel

C) O Knowledge

D) O e-mail de confirmação

B. A tela orienta; o banco decide.

4. Por que um Supabase novo, e não o da estante?

A) O Free obriga

B) Apps diferentes, com usuários e regras diferentes

C) Por causa da região

D) Para ter mais créditos

B. E compartilhar sobrescreveria os segredos de um app no outro.

○ RECAPITULANDO

A redação *está aberta*

Criamos o Supabase meu-blog em São Paulo, o projeto já conectado, sem Cloud, e o Knowledge com as regras do módulo. Definimos a matriz de permissões, montamos login e perfis com gatilho e guardamos os papéis numa tabela que o app não altera, com uma função no banco para as políticas futuras. A Autora Teste recebeu o papel pelo painel. Bookmark "blog base v1".

meu-blog em sa-east-1

Sem Enable Cloud

Matriz de permissões

papeis separado

Função tem_papel

Três contas

Próxima aula → [Página de criação de artigos](#)

Fontes: docs.lovable.dev (Connect to Supabase · Knowledge · Plan mode) e supabase.com/docs (Row Level Security · Managing user data · Password-based Auth · Redirect URLs · pricing), consultados em setembro de 2026. As telas são reproduções ilustrativas feitas para o curso, com contas de teste.